

An Extension of the L^AT_EX theorem environment*

Frank Mittelbach

November 12, 2021

This file is maintained by the L^AT_EX Project team.
Bug reports can be opened (category `tools`) at
<https://latex-project.org/bugs.html>.

Abstract

The macros described in this paper yield an extension of the L^AT_EX theorem mechanism. It is designed to satisfy the different requirements of various journals. Thus, the layout of the “theorems” can be manipulated by determining a “style”. This article describes not only the use, but also the definition, of the necessary macros.

Preface to version 2.2

For L^AT_EX 2_ε this package did not need any fundamental changes. I only modified the messages generated so that theorem layout styles will show up with the `\listfiles` command and cleaned the section on the New Font Selection Scheme since this is now included in L^AT_EX.

Preface to version 2.1

This version is identical to 2.0g described in *TUGboat* 10#3 except for some internal defaults which are now set depending on the used font selection scheme.

This was done to avoid unpleasant surprises if the new font selection scheme is in force. For further details see section 3 and [1].

1 Introduction

For our purposes here, “theorems” are labelled enunciations, often set off from the main text by extra space and a font change. Theorems, corollaries, conjectures, definitions, and remarks are all instances of “theorems”. The “header” of these structures is composed of a label (such as `THEOREM` or `REMARK`) and a number which serializes an item in the sequence of items with the same label.

Shortly after the introduction of L^AT_EX at the Fachbereich Mathematik in Mainz, the desire to manipulate the layout of “theorems” arose. In Mainz, the following two conventions came into general use:

*This file has version number v2.2c, last revised 2014/10/28.

1. The number of the theorem is shown in the margin.
2. There is a line break at the end of the theorem header.

Additionally, some journals require different formats which depend on the “sort of theorem”: e.g. often remarks and definitions are set in `\upshape`, while `\itshape` is employed for main theorems.

Confronted with these requirements, a theorem environment was developed in Mainz which allows separate determination of the layout of the “theorems sets”, comparable to `\pagestyle`.

2 The user interface

2.1 Defining new theorem sets

`\newtheorem` As in the original L^AT_EX version, the command `\newtheorem` defines a new “theorem set” or “theorem-like structure”. Two required arguments name the new environment and give the text to be typeset with each instance of the new “set”, while an optional argument determines how the “set” is enumerated:

`\newtheorem{foo}{bar}` The theorem set `foo` (whose name is `bar`) uses its own counter.

`\newtheorem{foo2}[foo]{bar2}` The theorem set `foo2` (printed name `bar2`) uses the same counter as the theorem set `foo`.

`\newtheorem{foo3}{bar3}[section]` The theorem set `foo3` (printed name `bar3`) is enumerated within the counter `section`, i.e. with every new `\section` the enumeration begins again with 1, and the enumeration is composed from the section-number and the theorem counter itself.

`\theoremstyle` Additionally, the command `\theoremstyle` can define the layout of various, or all, theorem sets. It should be noted that any theorem set defined by `\newtheorem` is typeset in the `\theoremstyle` that is current at the time of the definition. Thus, the following

<code>\theoremstyle{break}</code>	<code>\newtheorem{Cor}{Corollary}</code>
<code>\theoremstyle{plain}</code>	<code>\newtheorem{Exa}{Example}[section]</code>

leads to the result that the set `Cor` is formatted in the style `break`, while the set `Exa` and all the following ones are formatted in the style `plain`, unless another `\theoremstyle` follows. Since the definitions installed by `\newtheorem` are global, one also can limit `\theoremstyle` locally by grouping braces.

`\theorembodyfont` The choice of the font for the theorem body is completely independent of the chosen `\theoremstyle`; this has proven to be very advantageous. For example,

<code>{\theorembodyfont{\upshape}</code>	<code>\newtheorem{Rem}{Remark}}</code>
--	--

defines a theorem set `Rem`, which will be set in `\upshape` in the current layout (which in our example is `plain`). As with `\theoremstyle`, the `\theorembodyfont` chosen is that current at the time of `\newtheorem`. If `\theorembodyfont` is not specified or one defines `\theorembodyfont{}`, then the font used will be that defined by the `\theoremstyle`.

<code>\theoremheaderfont</code> <code>\theorempreskipamount</code> <code>\theorempostskipamount</code>	It is also possible to customize the font used for the theorem headers. This is, however, a global declaration, and therefore there should be at most one <code>\theoremheaderfont</code> declaration in the preamble.¹ Two additional parameters affect the vertical space around the theorem environments: <code>\theorempreskipamount</code> and <code>\theorempostskipamount</code> define, respectively, the spacing before and after such an environment. These parameters apply for all theorem sets and can be manipulated with the ordinary length macros. They are rubber lengths, ('skips'), and therefore can contain <code>plus</code> and <code>minus</code> parts. Since the definition of theorem sets should—most sensibly—be placed in the preamble, we only allow installation there. It is therefore possible to release the memory used here after <code>\begin{document}</code>, in order to make room for other applications.
--	--

2.2 Existing theorem styles

The following theorem styles exist to date:

<code>plain</code>	This theorem style emulates the original L ^A T _E X definition, except that additionally the parameters <code>\theorem...skipamount</code> are used.
<code>break</code>	In this style, the theorem header is followed by a line break.
<code>marginbreak</code>	The theorem number is set in the margin, and there is a line break as in <code>break</code> .
<code>changebreak</code>	Like <code>break</code> , but with header number and text interchanged.
<code>change</code>	Header number and text are interchanged, without a line break.
<code>margin</code>	The number is set in the left margin, without a line break.

All styles (except `plain`) select `\slshape` as the default `\theorembodyfont`.

2.3 Examples

Given the above theorem sets `Cor`, `Exa` and `Rem`, suppose that the preamble also contains the declarations:

```

\theoremstyle{marginbreak} \newtheorem{Lem}[Cor]{Lemma}
\theoremstyle{change}
\theorembodyfont{\itshape} \newtheorem{Def}[Cor]{Definition}

\theoremheaderfont{\scshape}

```

Then the following are some typical examples of the typeset output resulting from their use.

COROLLARY 1

This is a sentence typeset in the theorem environment Cor.

EXAMPLE 2.1 *This is a sentence typeset in the theorem environment Exa.*

¹If it is actually necessary to have different header fonts, one has to define new theorem styles (substituting the desired font) or specify the information directly in the `\newtheorem` declaration (the `unclean` variant).

REMARK 1 This is a sentence typeset in the theorem environment `Rem`.

2 LEMMA (BEN USER)

This is a sentence typeset in the theorem environment `Lem`.

3 DEFINITION (VERY IMPRESSIVE DEFINITION) *This is a sentence typeset in the theorem environment `Def`.*

The last two examples show the effect of the optional argument to a theorem environment (it is the text typeset in parentheses).

3 Special Considerations

Theoremheader and body are implemented as a unit. This means that the `\theoremheaderfont` will inherit characteristics of the `\theorembodyfont` in $\text{\LaTeX 2}_{\epsilon}$. Thus, if for example `\theorembodyfont` is `\itshape` and `\theoremheaderfont` is `\bfseries` the font selected for the header will have the characteristics ‘bold extended italic’. If this is not desired one should set the `\theoremheaderfont` to something like

```
\theoremheaderfont{\normalfont\bfseries}
```

i.e. supplying all necessary font information explicitly.

4 Acknowledgements

The publication of this set of macros was only possible with the help of Christina Busse (translating the manuscript into English), Joachim Pense (playing the rôle of typist), Chris Rowley (looking everything over) and many others providing useful suggestions.

5 The documentation driver file

The next bit of code contains the documentation driver file for $\text{\TeX}$, i.e., the file that will produce the documentation you are currently reading. It will be extracted from this file by the `docstrip` program. Since it is the first code in the file one can alternatively process this file directly with $\text{\LaTeX 2}_{\epsilon}$ to obtain the documentation.

```
1 (*driver)
2 \documentclass{ltxdoc}
3
4 \usepackage{theorem}
5
6 <+driver>% The next few lines define theorem sets which are used
7 <+driver>% in the example section of the documentation.
8
9 \theoremstyle{break}          \newtheorem{Cor}{Corollary}
10 \theoremstyle{plain}         \newtheorem{Exa}{Example}[section]
11 {\theorembodyfont{\upshape}\newtheorem{Rem}{Remark}}
12 \theoremstyle{marginbreak} \newtheorem{Lem}{Cor}[Lemma]
13 \theoremstyle{change}
```

```

14 \theorembodyfont{\itshape} \newtheorem{Def}[Cor]{Definition}
15
16 \theoremheaderfont{\scshape}
17
18 \RecordChanges
19
20 \begin{document}
21   \DocInput{theorem.dtx}
22 \end{document}
23 \>/driver>

```

6 Definition of the Macros

If the file has been loaded before, we abort immediately. If not the package announces itself (this is actually done at the very top if the file—the way it is done isn’t good style so don’t copy it).

```

24 (*package)
25 %\@ifundefined{theorem@style}{-}{\endinput}
26 %\def\FMithmInfo{1995/11/19 v2.2b Theorem extension package (FMi)}
27 %\ProvidesPackage{theorem}[\FMithmInfo]

```

6.1 Definition of theorem styles and fonts

All the definitions in this file are done globally to allow inputting this file inside a group.

`\theoremstyle` Before a theorem style can be installed, the chosen style must be known. For that reason, we must test to see that `\th@<style>` is known or, more precisely, that it is different from `\relax`. If the style is not known then `\th@plain` is used.

```

28 \gdef\theoremstyle#1{%
29   \@ifundefined{th@#1}{\@warning
30     {Unknown theoremstyle ‘#1’. Using ‘plain’}%
31     \theoremstyle{plain}}%

```

We save the theorem style to be used in the token register `\theorem@style`.

```

32   {\theorem@style{#1}}%

```

Now we “evaluate” the theorem style: this means, we call the macro `\th@<style>` which will activate the relevant definitions which are contained in a separate file. This is done in a group to suppress changes to the current font. This could otherwise pose problems together with the new font selection scheme² if the `\th@<style>` is evaluated a second time.

```

33   \begingroup
34     \csname th@\the\theorem@style \endcsname
35   \endgroup

```

`\@begintheorem` We reset `\@begintheorem` and `\@opargbegintheorem` to `\relax` since these commands are no longer necessary at toplevel. This will save a few tokens.

```

36 \global\let\@begintheorem\relax
37 \global\let\@opargbegintheorem\relax

```

²When I printed the original article using the new font selection scheme I ended with a document with slanted typefaces (text headings and all) simply because one of the theorem styles used `\sl` at toplevel.

`\theorem@style` Obviously the token register used above has to be allocated. To assure the utmost compatibility with the original $\text{\LaTeX}$ definition, we set the default theorem style to `plain`, which implements the usual $\text{\LaTeX}$ convention.

```

38 \newtoks\theorem@style
39 \global\theorem@style{plain}

```

`\theorembodyfont` If the `\theorembodyfont` is set by the user then it should not interact with the default font set in the theorem style. When the new font selection is in force this may happen if, for example, the default is `\itshape` and the new `\theorembodyfont` is `\sffamily`. So we add a `\reset@font` command in front of the user definition.

```

40 \gdef\theorembodyfont#1{%

```

We check if the argument supplied is empty and if so put nothing into the `\theorem@bodyfont` token register to allow for `\theorembodyfont{}` as a mean of using the default of the current `\theoremstyle`.

```

41   \def\@tempa{#1}%
42   \ifx\@tempa\@empty
43     \theorem@bodyfont{}%
44   \else
45     \theorem@bodyfont{\reset@font#1}%
46   \fi
47 }
48 \newtoks\theorem@bodyfont
49 \global\theorem@bodyfont{}

```

`\theoremheaderfont` The font for the theorem headers is handled differently because this definition applies to all theorem styles.

```

50 \gdef\theoremheaderfont#1{\gdef\theorem@headerfont{#1}%

```

After using the macro once it is redefined to produce an error message.

```

51   \gdef\theoremheaderfont##1{%
52     \typeout{\string\theoremheaderfont\space should be used
53             only once.}}

```

`\theorem@headerfont` To set the `\theorem@headerfont` default we first test if the new fontselection scheme is in force.

```

54 \ifx\upshape\undefined

```

If not we define it to expand into `\bfseries`. We don't use `\let` just in case a following style option redefines this macro.

```

55 \gdef\theorem@headerfont{\bfseries}

```

Otherwise we reset the current shape before calling `\bfseries`.

```

56 \else \gdef\theorem@headerfont{\normalfont\bfseries}\fi

```

`\th@plain` The different styles are defined in macros such as `\th@plain`. Since memory space is precious in “non-Big-versions”, we have to avoid offering too many unused definitions. Therefore we define these styles in separate files that can be loaded on demand. Thus the commands themselves only load these files. We use `\th@change` a $\text{\LaTeX}$ 2_{ϵ} internal command that ensures that the file will be listed with `\listfiles`

```

57 \gdef\th@plain{\input{thp.sty}}

```

```

58 \gdef\th@break{\@input@{thb.sty}}
59 \gdef\th@marginbreak{\@input@{thmb.sty}}
60 \gdef\th@changebreak{\@input@{thcb.sty}}
61 \gdef\th@change{\@input@{thc.sty}}
62 \gdef\th@margin{\@input@{thm.sty}}

```

This list will be expanded when new styles become available. For testing, just append new theorem substyles as document options.

6.2 Definition of a new theorem set

As already pointed out, a new theorem environment can be defined in three different ways:

```

\newtheorem{Lem}{Lemma}
\newtheorem{Lem}{Lemma}[section]
\newtheorem{Lem}[Theorem]{Lemma}

```

The function of the macro `\newtheorem` is to recognize these cases and then to branch into one of the three macros `\xynthm`, `\xnthm` or `\othm`. This mechanism is adopted unchanged from [2]; the essential point here is that, for example, in the second case, the arguments `Lem`, `Lemma` and `section` are passed over to the macro `\xnthm`.

We inspect this case first because the others present fewer problems, and thus are easily derived from this one.

`\xnthm` For our example arguments, the macro `\xnthm` must fulfill the following:

- Define a new L^AT_EX-counter ‘Lem’
- reset this counter within a `\section`
- define the macro `\theLem`
- define the environment macros `\Lem` and `\endLem` using the current `\theoremstyle` and `\theorem@bodyfont`.

Obviously, all this should happen only if the first argument of `\xnthm` (i.e. `Lem` in our example) is chosen so as not to conflict with any previously defined commands or environments. This test is performed by the L^AT_EX macro `\ifdefinable`.

```

63 \gdef\xnthm#1#2[#3]{\expandafter\ifdefinable\csname #1\endcsname

```

Therefore, the first argument of `\ifdefinable` is the expansion (in the example, `\Lem`) of `\csname#1\endcsname`. The second argument is executed only if the test has been completed successfully.

```

64   {%

```

Now we define the new counter. The names of the L^AT_EX macros employed should speak for themselves:

```

65   \definecounter{#1}\newctr{#1}[#3]%

```

Using `\newctr` will give a proper error message if the counter in `#3` is not defined. In defining ‘`\theLem`’ we must generate the desired macro name by use of `\expandafter` and `\csname`.

```

66   \expandafter\xdef\csname the#1\endcsname

```

An `\xdef` is used in order to make the definition global, and to ensure that it contains the replacement texts of `\@thmcountersep` and `\@thmcounter`.³ However, not everything should be expanded. For example, it saves space to use `\thesection` instead of its—at times—lengthy expansion.

```
67      {\expandafter \noexpand \csname the#3\endcsname
68       \@thmcountersep \@thmcounter{#1}}%
```

Thus with the defaults of L^AT_EX, `\theLem` would be replaced by the command sequence `\thesection.\arabic{Lem}`.

We will now look at the definition of the macro which is executed at the beginning of the actual environment (in our example this macro is `\Lem`). It should be noted that we use an “`\expandafter` trick” to expand only certain parts of the replacement text at the time of the definition.

```
69      \def\@tempa{\global\@namedef{#1}}%
70      \expandafter \@tempa \expandafter{%
```

First, the macro that contains the current definitions of `\@begintheorem` and `\@opargtheorem` should be called up. The name of this macro—as is already known—has the form `\th@<theorem style>`; therefore, it must be called by

```
71      \csname th@\the \theorem@style
```

In addition the default theorem font should be changeable, i.e. we have to insert the contents of `\theorem@bodyfont`. For that reason, we expand even further, beyond `\endcsname`, and thus insert the contents of the token register `\theorem@bodyfont` in the replacement text.

```
72      \expandafter \endcsname \the \theorem@bodyfont
```

Now it is time to call the macro `\@thm` which takes over the further processing. It has two arguments: the current counter name (in our example, `Lem`), and the text of the label (in our example, `Lemma`).

```
73      \@thm{#1}{#2}}%
```

With this, the ‘sub-definition’ is complete. The macro `\@endtheorem` ends a theorem environment and is, so far, nothing but an `\endtrivlist`. (Hence it is defined globally, and not within the theorem styles.⁴) Therefore, we can set it equivalent to the macro that ends the theorem set (in our example, `\endLem`). However, if some day theorem styles exist that do change `\@endtheorem`, we would have to use the commented-out line instead.

```
74      \global \expandafter \let \csname end#1\endcsname \@endtheorem
75 %      \global\@namedef{end#1}{\@endtheorem}%
```

With these commands all the required definitions are employed, unless the test `\@ifdefinable` has failed. Therefore, we end the second argument of this macro and with it the definition of `\@xnthm`.

```
76      }}
```

`\@ynthm` The definition of `\@ynthm` is completely analogous. In this case the new counter that is defined is not reset within another counter; thus the definition of `\the...` is simplified:

```
77 \gdef\@ynthm#1#2{\expandafter\@ifdefinable\csname #1\endcsname
```

³These two macros can be defined by the document style. Their default values produce a ‘.’ as separation and an arabic representation of the number.

⁴This has to be changed as soon as theorem styles that change `\@endtheorem` exist. In such a case, all existing styles must be changed as well since they will have to reset the macro.


```

78   {\@definecounter{#1}%
79     \expandafter\xdef\csname the#1\endcsname{\@thmcounter{#1}}%
The rest of the definition corresponds literally to that of \@xnthm:
80   \def\@tempa{\global\@namedef{#1}}\expandafter \@tempa
81     \expandafter{\csname th@\the \theoremstyle \expandafter
82       \endcsname \the\theorem@bodyfont \@thm{#1}{#2}}%
83   \global \expandafter \let \csname end#1\endcsname \@endtheorem}}

```

`\@othm` The definition of `\@othm` does not contain anything new.

```

84 \gdef\@othm#1[#2]#3{%
We do not define a new counter but instead use one that has already been defined.
Thus the only definition we need is that of this pseudo-counter (i.e. \the<env.
name>)). First we check if #2 corresponds to a known counter name.
85   \expandafter\ifx\csname c#2\endcsname\relax
86     \@nocounterr{#2}%
87   \else
88     \expandafter\ifdefinable\csname #1\endcsname
89     {\expandafter \xdef \csname the#1\endcsname
90       {\expandafter \noexpand \csname the#2\endcsname}}%

```

All other parts of the definition can be adopted from `\@xnthm`. We have to remember, though, that in this case the name of the current counter and the theorem label have moved to the second and third arguments.

```

91   \def\@tempa{\global\@namedef{#1}}\expandafter \@tempa
92     \expandafter{\csname th@\the \theoremstyle \expandafter
93       \endcsname \the\theorem@bodyfont \@thm{#2}{#3}}%
94   \global \expandafter \let \csname end#1\endcsname \@endtheorem}%
95   \fi}

```

6.3 Macros that are employed in a theorem environment

`\@thm` The macro `\@thm` has to increase the current counter. Then, depending on whether the environment has (or does not have) an optional argument, it has to branch into either `\@begintheorem` or `\@opargtheorem`.

```

96 \gdef\@thm#1#2{\refstepcounter{#1}%

```

Now we start a `trivlist` environment, and give `\@topsep` and `\@topsepadd` the values of the skip registers `\theorempreskipamount` and `\theorempostskipamount`. The value in `\@topsep` is the vertical space that is inserted by the first (and only) `\item` in our `\trivlist` whilst `\@topsepadd` is inserted by `\@endparenv` at the end of that `trivlist` environment. By using these registers, we obtain the desired space around a `theorem` environment.

```

97   \trivlist
98     \@topsep \theorempreskipamount           % used by first \item
99     \@topsepadd \theorempostskipamount       % used by \@endparenv

```

Now we have to test whether an optional argument has been given.

```

100   \ifnextchar [%

```

If there is an optional argument, we will call `\@ythm`, and move the arguments read back into the input stream.

```

101   {\@ythm{#1}{#2}}%

```

If not, we call `\@begintheorem`. Its first argument is the name of the theorem set (hence the second argument of `\@thm`). Its second argument is the macro that produces the current number.

```
102 {\@begintheorem{#2}{\csname the#1\endcsname}\ignorespaces}}
```

`\@xthm` Both these macros were originally called by `\@thm`. We do not need `\@xthm` anymore, hence we reset it to `\relax`. The definition of `\@ythm` has not changed at all from its definition in L^AT_EX. In order to make the macros easier to understand, we will nevertheless present it (commented out).

```
103 \global\let\@xthm\relax
104 % \def\@ythm#1#2[#3]{\@opargbegintheorem{#2}{\csname
105 % the#1\endcsname}{#3}\ignorespaces}
106 \endpackage
```

The primitive `\ignorespaces` in `\@ythm` and `\@thm` is needed to remove the spaces between the `\begin{...}` and the actual text.

6.4 Definition of the theorem substyles

As already pointed out, the theorem substyles, defined below, are only loaded when necessary. Note that all these substyles, except `plain`, have `\slshape` as the default body font.

6.4.1 The plain style

As the following macros use `@`, we have to locally set the `\catcode` of this symbol to “letter”. This happens within a group, so that we do not have to worry about which `\catcode` that symbol had before.

```
107 (*thp)
108 \begingroup \makeatletter
```

Since we are now within a group, we must make all definitions globally. First we make sure that `theorem.sty` is loaded. This will allow us to use this file as a document style option without having to call `theorem` itself as an option. At the same time, we assure that at least version 2 is loaded, since `\theorem@style` was not defined in earlier versions.

```
109 \@ifundefined{theorem@style}{\input{theorem.sty}}{}
110 \ProvidesFile{thp.sty}
111 [\FMithmInfo]
```

`\th@plain` `\theoremstyle{plain}` corresponds to the original definition, except that the distances to the surrounding text are determined by the parameters `\theorempreskipamount` and `\theorempostskipamount`. First we set the default body font.

```
112 \gdef\th@plain{\normalfont\itshape
```

Then we define `\@begintheorem` and `\@opargbegintheorem`. These two macros define how the header of a theorem is typeset. `\@opargbegintheorem` will be called if a `theorem` environment with an optional argument is encountered; otherwise, the header is constructed by calling `\@begintheorem`. If one of these macros is executed, we are within a `trivlist` environment started by `\@thm`. So the theorem header is produced with an `\item` command.

Instead of specifying the header font directly, all standard theorem styles use the `\theorem@headerfont` macro to allow customization. The extra space (`\labelsep`) is necessary because of problems in the `trivlist` environment.

```
113 \def\@begintheorem##1##2{%
114     \item[\hskip\labelsep \theorem@headerfont ##1\ ##2]}%
The definition of \@opargbegintheorem is completely analogous. The only difference is the fact that there exists a third argument (which is the optional parameter of the environment and contains additional information about the theorem). Customarily we enclose it in parentheses.
115 \def\@opargbegintheorem##1##2##3{%
116     \item[\hskip\labelsep \theorem@headerfont ##1\ ##2\ (##3)]}}
```

We conclude with an `\endgroup` to restore the `\catcode` of `@`.

```
117 \endgroup
118 \thp
```

6.4.2 The break style

This style option is stored in the file `thb.sty`. For the next two lines see the documentation for `\th@plain` on page 10.

```
119 (*thb)
120 \begingroup \makeatletter
121 \ifundefined{theorem@style}{\input{theorem.sty}}{}
122 \ProvidesFile{thb.sty}
123     [\FMithmInfo]
```

`\th@break` `\theoremstyle{break}` produces a line break after the name of the theorem. The font is `\slshape`. Hence, we define `\th@break` as follows:

```
124 \gdef\th@break{\normalfont\slshape
125 \def\@begintheorem##1##2{\item[%
```

We run into the following problem: it is not possible to create the header with `\item[<title>]` and then start a new line by, for example, `\mbox{}\\`. Such a definition will fail whenever a list environment follows immediately. With the above construction, the `\mbox{}` causes the switch `@inlabel` (cf. definition of `\list` and `\trivlist` in [2]) to be set to `false` and so the following list will insert additional vertical space (`\topskip`). This is quite annoying. Therefore, we create the line break within the `\item`. In order to ensure that the text will begin at the proper position in the following line, we simply pretend that the label does not take any room.⁵

```
126     \rlap{\vbox{\hbox{\hskip \labelsep \theorem@headerfont ##1\ ##2}%
127         \hbox{\strut}}}}}%
Again, the definition of \@opargbegintheorem is completely analogous.
```

```
128 \def\@opargbegintheorem##1##2##3{%
129     \item[\rlap{\vbox{\hbox{\hskip \labelsep \theorem@headerfont
130         ##1\ ##2\ (##3)}%
131         \hbox{\strut}}}}}]
132 \endgroup
133 \thb
```

⁵This will lead to problems whenever very high symbols occurring in the line tower into the heading. So, something else has to be done here sometime.

6.4.3 The changebreak style

```

134 % This style option is stored in the file |thcb.sty|.
135 % \begin{macrocode}
136 (*thcb)
137 \begingroup \makeatletter
138 \@ifundefined{theorem@style}{\input{theorem.sty}}{}
139 \ProvidesFile{thcb.sty}
140 [\FMithmInfo]

\th@changebreak The change-break theorem style is like break but with interchange of theorem
name and theorem number. Thus we define \th@changebreak as follows:

141 \gdef\th@changebreak{\normalfont\slshape
142 \def\@begintheorem##1##2{\item
143 [\rlap{\vbox{\hbox{\hspace\labelsep \theorem@headerfont ##2\ ##1}}%
144 \hbox{\strut}}}}}%
145 \def\@opargbegintheorem##1##2##3{%
146 \item[\rlap{\vbox{\hbox{\hspace\labelsep \theorem@headerfont
147 ##2\ ##1\ (##3)}}%
148 \hbox{\strut}}}}}%
149 \endgroup
150 \end{thcb}

```

6.4.4 The change style

This style option is stored in the file thc.sty.

```

151 (*thc)
152 \begingroup \makeatletter
153 \@ifundefined{theorem@style}{\input{theorem.sty}}{}
154 \ProvidesFile{thc.sty}
155 [\FMithmInfo]

\th@change The change theorem style corresponds to the change break style without a line-
break after the header. To say it in another way, it's the same as the plain style
but with number and name interchanged and \slshape as the default font.

156 \gdef\th@change{\normalfont\slshape
157 \def\@begintheorem##1##2{\item
158 [\hspace\labelsep \theorem@headerfont ##2\ ##1]]}%
159 \def\@opargbegintheorem##1##2##3{%
160 \item[\hspace\labelsep \theorem@headerfont ##2\ ##1\ (##3)]}%
161 \endgroup
162 \end{thc}

```

6.4.5 The marginbreak style

This style option is the one used most often at Mainz. It is saved in the file thmb.sty.

```

163 (*thmb)
164 \begingroup \makeatletter
165 \@ifundefined{theorem@style}{\input{theorem.sty}}{}
166 \ProvidesFile{thmb.sty}
167 [\FMithmInfo]

```

`\th@marginbreak` The `margin break` style is nearly the same as the `change break` style. The only difference is the placement of the theorem number. We use `\llap` to place it in the left margin.

In this style `\labelsep` denotes the separation between the number and the text.

```

168 \gdef\th@marginbreak{\normalfont\slshape
169   \def\@begintheorem##1##2{\item
170     [\rlap{\vbox{\theorem@headerfont
171       \hbox{\llap{##2}\hskip\labelsep ##1}%
172       \hbox{\strut}}}}]}%
173 \def\@opargbegintheorem##1##2##3{%
174   \item[\rlap{\vbox{\theorem@headerfont
175     \hbox{\llap{##2}\hskip\labelsep ##1\ (#3)}%
176     \hbox{\strut}}}}]}%
177 \endgroup
178 \thmb)

```

6.4.6 The margin style

This style option is stored in the file `thm.sty`.

```

179 (*thm)
180 \begingroup \makeatletter
181 \ifundefined{theorem@style}{\input{theorem.sty}}{}
182 \ProvidesFile{thm.sty}
183     [\FMthmInfo]

```

`\th@margin` Again this is only a variant of the theorem styles described above without any new ideas.

```

184 \gdef\th@margin{\normalfont\slshape
185   \def\@begintheorem##1##2{\item
186     [\theorem@headerfont \llap{##2}\hskip\labelsep ##1]}%
187 \def\@opargbegintheorem##1##2##3{%
188   \item[\theorem@headerfont \llap{##2}\hskip\labelsep ##1\ (#3)]}%
189 \endgroup
190 \thm)

```

6.5 Final Definitions

`\theorempreskipamount` The `skip` parameters that regulate the vertical empty space before and after the theorem environment have to be allocated as well.

```

191 (*package)
192 \newskip\theorempreskipamount
193 \newskip\theorempostskipamount

```

Since we have used the same values for all theorem sets, we now can assign them.

```

194 \global\setlength\theorempreskipamount{12pt plus 5pt minus 3pt}
195 \global\setlength\theorempostskipamount{8pt plus 3pt minus 1.5pt}

```

`\@endtheorem` The same holds for the macro `\@endtheorem`, which ends a `theorem` environment. Since it is the same for all theorem sets, it is removed from the macros `\th@{style}`. It simply ends the `trivlist` environment, which was begun in `\@thm`.

```

196 \global\let\@endtheorem=\endtrivlist

```

`\@preamblecmds` All macros defined above are to be used only in the preamble. Therefore, we insert them in `\@preamblecmds` which will disable them at begin document. This is done by the internal $\text{\LaTeX} 2_{\epsilon}$ command `\@onlypreamble`.

```
197 \@onlypreamble\xnthm
198 \@onlypreamble\ynthm
199 \@onlypreamble\othm
200 \@onlypreamble\newtheorem
201 \@onlypreamble\theoremstyle
202 \@onlypreamble\theorembodyfont
203 \@onlypreamble\theoremheaderfont
```

Finally we declare the `plain` theorem style to be the default.

```
204 \theoremstyle{plain}
205 \</package>
```

References

- [1] M. GOOSSENS, F. MITTELBACH and A. SAMARIN. The $\text{\LaTeX}$ Companion. Addison-Wesley, Reading, Massachusetts, 1994.
- [2] LAMPORT, LESLIE. `latex.tex`, version 2.09, date Feb. 1990.

Change History

v1.0	won't work.	1
General: First draft.	1	v2.0b
v1.0a	General: 'global' used for all definitions.	1
General: 'thistheoremstyle' added.	1	
v1.1a	v2.0c	
General: New concept: redefine the internal $\text{\LaTeX}$ commands '@xnthm' etc.	1	General: Documentation translated to English. Thanks to LLL and Nina
v1.1b	v2.0d	
General: 'theoremfont' used to specify font.	1	<code>\@thm</code> : Forgotten percent added.
v1.2a	v2.0e	
General: '@endtheorem' now globally defined.	1	General: Spaces removed from 'typeout'.
v1.2b	v2.0f	
General: 'newtheorem' only allowed in the preamble.	1	<code>\theorem@bodyfont</code> : Unnecessary 'global' removed.
v1.3a	<code>\theorem@style</code> : Unnecessary 'global' removed.	6
General: 'theoremfont' now a toks register.	1	<code>\theorem@postskipamount</code> : Unnecessary 'global' removed.
v2.0a		13
General: New implementation and new documentation.	1	v2.0g
No layout changes but internal modifications old substyles		<code>\@preamblecmds</code> : 'theoremheaderfont' and 'theorembodyfont' added.
		14

<code>\theorem@bodyfont</code> : ‘theoremfont to ‘theorembodyfont.	6	v2.1c <code>\th@marginbreak</code> : ‘endgroup outside macrocode env.	13
<code>\theorem@headerfont</code> : Macro added.	6	<code>\theorem@bodyfont</code> : ‘reset@font added and logic changed.	6
<code>\theoremheaderfont</code> : Macro added.	6	v2.1d <code>\theorem@bodyfont</code> : Reset to empty if arg is empty	6
v2.1a <code>\@begintheorem</code> : Relaxed at toplevel.	5	v2.2a <code>\@preamblecmds</code> : Use L ^A T _E X 2 _ε interface	14
<code>\@opargbegintheorem</code> : Relaxed at toplevel.	5	<code>\th@margin</code> : Use ‘@input@ to load theorem layout files	6
<code>\theorem@headerfont</code> : Support for new font selection.	6	General: Cleanup for L ^A T _E X2e . . .	1
<code>\theoremstyle</code> : Group added around ‘th@..	5	v2.2b <code>\@othm</code> : Build in check for counter defined	9
Reseting of ‘..@begintheorem removed.	5	<code>\@xnthm</code> : Use <code>\@newctr</code>	7
General: Support for new font selection.	1	General: Announce moved to top of file	5
v2.1b General: Fixed missing ‘item by declaring default style.	14	v2.2c <code>\@xnthm</code> : Use <code>\@newctr</code> properly. .	7